Sources

Treat raw data as a contract: schema, quality, freshness, and enforcement

Sources as a contract

The agreement a staging model is actually built against

```
sources:
  - name: raw
    database: analytics_raw
    schema: public
    tables:
      - name: orders
        loaded_at_field: _loaded_at
        freshness:
          warn_after: {count: 12, period: hour}
          error_after: {count: 24, period: hour}
```

The contract, four parts

Schema, quality, SLA, and the enforcement that makes it real

Schema	What columns, what types
Quality	Idempotent, no duplicates on rerun
SLA	The freshness block above
Enforcement	The gate below, not a wiki page

Staging models

One job only: rename and cast, never drop a row or a column

```
select
    id as order_id,
    customer_id,
    cast(amount_cents as numeric) as amount
from {{ source('raw', 'orders') }}
```

```
materialized: view
Not a table

Never drop a row or a column here
Metadata only
```

Generate with codegen

Reading information_schema beats asking an AI to guess

```
dbt run-operation generate_source
From raw schema

dbt run-operation generate_base_model
Staging skeleton

dbt run-operation generate_model_yaml
Properties file

Review and rename
Don't ship as-is
```

Idempotent loads

A retry or a backfill should never double-count a row

```
Blind append
Duplicates on retry/rerun

Merge on a key (order_id)
Rerun is a no-op

Or a surrogate key
When there's no natural one
```

Freshness config fields

What actually defines the SLA half of the contract

```
warn_after: {count, period}
Soft threshold

error_after: {count, period}
Hard threshold

loaded_at_field
Timestamp column to check

filter
Limit how much gets scanned
```

Source freshness

v2 checks model freshness too, not just sources

```
dbt source freshness
dbt v1, sources only

dbt freshness
dbt v2, sources + models
```

Freshness as a run gate

The enforcement that makes the first three sections real

```
dbt build --select "source_status:fresher+"
Not everything

dbt source freshness && dbt build
Gate, then build
```